Step 2a — Dataset: unire le fonti esistenti

Obiettivo

Trasformare dataset pubblici molto diversi fra loro (NLI, fact checking su Wikipedia, riassunti, claim reali) in **un unico formato**: coppie (documento in prosa, claim in prosa, etichetta binaria), pulite, decontaminate rispetto ai benchmark di test. La divisione in train / dev / test senza documenti in comune è descritta allo Step 0.

Input e output

Input	file in `Datasets/` (FEVER, HoVer, FactKG, AVeriTeC), cache HF di FEVER, dataset scaricati da Hugging Face (MiniCheck, ANLI, VitaminC, SummEdits, SciFact, WiCE, RAGTruth)
Output	`data/finetune/raw/<fonte>.jsonl` (una riga per coppia) e `<fonte>.stats.json`
Codice	`data_prep/build_pairs.py` (esiste, testato su tutte le fonti)

1. Il formato comune

Ogni riga, qualunque sia la fonte:

```
{"id": "anli:train:0fd0abfb-659e-4453-b196-c3a64d2d8267",
 "source": "anli", "split": "train",
 "document": "The Parma trolleybus system (Italian: \"Rete filoviaria di Parma\") forms part of the
  ↪  public transport network of the city and \"comune\" of Parma, in the region of Emilia-Romagna,
  ↪  northern Italy. In operation since 1953, the system presently comprises four urban routes.",
 "claim": "The trolleybus system has over 2 urban routes",
 "label": 1, "orig_label": "entailment",
 "doc_id": "b03f7d2b59d9d58a", "meta": {"round": "r1"}}
```

Campo	Perché serve
`label`	1 = supportato, 0 = contraddetto o non stabilito: la convenzione di LLM-AggreFact
`orig_label`	l'etichetta originale a 3 classi quando esiste (SUPPORTS / REFUTES / NOT ENOUGH INFO): serve a costruire i target delle tre domande (03_rendering_formato.md)
`doc_id`	chiave di gruppo: righe con lo stesso documento finiscono sempre nello stesso split
`meta`	informazioni utili per analisi e ablazioni (tipo di ragionamento, dominio, troncamento)

2. Come si costruisce il documento per ogni fonte

Il punto delicato: molti dataset di fact checking **non hanno un documento**, hanno un riferimento all'evidenza (pagina e numero di frase di Wikipedia). Per evitare il retrieval, l'evidenza gold si "materializza" una volta, qui, in un documento.

Fonte	Da cosa nasce il documento	Mappatura dell'etichetta	Coppie (train / dev)
MiniCheck C2D/D2C	già presente (documenti sintetici o notizie)	1 / 0	14.395 / —
ANLI (R1–R3)	la premessa	entailment → 1; neutral, contradiction → 0	162.865 / 3.200

Fonte	Da cosa nasce il documento	Mappatura dell'etichetta	Coppie (train / dev)
SummEdits	il documento originale; il claim è il riassunto (originale o modificato)	consistente → 1	6.348 / —
HoVer	testo introduttivo di **tutte** le pagine Wikipedia di evidenza (2–4), in ordine casuale	SUPPORTED → 1; NOT_SUPPORTED → 0	17.000 / 3.745
FEVER	pagine intere (fino a 40 frasi) della prima evidenza gold, non solo la frase di evidenza; per NOT ENOUGH INFO, la pagina il cui titolo è il più lungo n-gramma del claim	SUPPORTS → 1; REFUTES, NEI → 0	144.411 / 9.926
VitaminC	`Title: <pagina>` + frase di evidenza	SUPPORTS → 1; altro → 0	370.653 / 63.054
AVeriTeC	coppie domanda–risposta di evidenza degli annotatori (con fonte), **senza** la giustificazione che rivelerebbe il verdetto	Supported → 1; Refuted, NEE → 0; Conflicting escluso	2.873 / 462
SciFact	titolo + abstract	SUPPORT → 1	919 / 340
WiCE	frasi della pagina web citata	supported → 1	3.470 / 949
RAGTruth	contesto (+ domanda); il claim è l'intera risposta	nessuna allucinazione → 1	15.090 / —
FactKG	**oggi**: lista di triple del grafo → **da non usare così**; va trasformato in prosa (02b)	True → 1	81.342 / 12.587

Perché il documento intero e non la sola frase di evidenza (FEVER, HoVer): in uso il modello riceverà testi lunghi in cui l'evidenza è una frase fra tante. Se in training vedesse solo la frase giusta, imparerebbe un compito più facile di quello reale.

Perché le pagine di HoVer in ordine casuale: l'ordine originale rispecchia la catena di ragionamento (hop 1, hop 2, ...) e il modello potrebbe sfruttarlo.

Esempi reali

SummEdits, coppia minima (stesso documento, due riassunti):

> Documento: "Ann: Darling, buy some butter. Mike: Ok. Mike: Do we have any milk? Ann: I will check... Ann: Yes, There is some in fridge."
>
> Claim A (label 1): "Ann **asks** Mike to buy butter and Mike confirms it. Later, Mike asks Ann if there is any milk, and after **checking**, Ann confirms that there is some in the fridge."
>
> Claim B (label 0): "Ann **orders** Mike to buy butter and Mike confirms it. Later, Mike asks Ann if there is any milk, and after **examining**, Ann confirms that there is some in the fridge."

È il tipo di errore sottile (attribuzione di un tono, "orders" invece di "asks") che i nostri modelli sbagliano sui riassunti. Mostra anche il limite: alcune modifiche sono al confine dell'opinabile, quindi SummEdits porta un po' di rumore.

VitaminC, evidenza contrastiva (stesso claim, due versioni della stessa pagina):

> Claim: "Manchester had a population of more than 540,000 in 2017 and was the 5th most populous English district."
>
> Evidenza 1 (label 1): "... with a population of **545,500 as of 2017** (**5th** most populous English district)."
>
> Evidenza 2 (label 0): "... with a population of **534,982 as of 2018** (**4th** most populous English district)."

Insegna a guardare numeri e date, non la somiglianza generale del testo.

3. Decontaminazione

Prima di usare una coppia si controlla che non si sovrapponga ai benchmark di valutazione. Oggi il controllo è contro LLM-AggreFact (dev + test); va esteso agli altri test della suite (06_valutazione.md).

Regola (in `make_mix.py`): scarto se il claim normalizzato coincide con un claim del benchmark, o se $\geq 20\%$ dei 13-grammi di parole del documento compare nei documenti del benchmark.

Risultati misurati (run completo):

Fonte	Scartate	Conseguenza
RAGTruth	**98%** dei documenti	esclusa: train e test condividono i documenti
WiCE	**30%** (13% sul solo train)	solo in un esperimento dedicato
VitaminC, FEVER, ANLI, MiniCheck, HoVer	$\leq 0{,}2\%$	trascurabile, righe tolte
FactKG, SummEdits, SciFact	0	

4. Pulizia e split

Fatti da `make_mix.py` (02e_mix_finale.md):

- **Duplicati** (stesso documento e stesso claim): 8.230 in FEVER, 3.760 in VitaminC.
- **Lunghezza**: documenti oltre 24.000 caratteri scartati.
- **Split per documento**: train, dev e test si costruiscono come descritto allo Step 0 (test ufficiali dove etichettati, altrimenti divisione per `doc_id`); le righe di train che condividono un `doc_id` con dev o test vengono tolte (oggi, per il solo val: 37.702 in ANLI, 4.826 in FEVER). Senza questo passo dev e test misurerebbero la memoria dei documenti, non la capacità di verifica.

5. Comandi

```
conda activate factcheck
python data_prep/build_pairs.py all                        # tutte le fonti (FEVER e FactKG: alcuni
↪ minuti)
python data_prep/build_pairs.py hover fever --limit 200    # prova veloce:
↪ data/finetune/raw/<fonte>.limit200.jsonl
```

I download da Hugging Face a volte falliscono con un errore 500 del proxy: basta rilanciare.

6. Controlli di qualità da fare a mano su un campione

Su 20 righe per fonte (5 minuti di lettura):

- il documento contiene davvero l'evidenza per le righe con label 1?
- per le righe con label 0, il motivo è chiaro (contraddizione o informazione mancante)?
- il documento è leggibile (niente markup, token tipo `-LRB-`)?

Il builder di FEVER già converte i token speciali (`-LRB-` → `(`) e ripulisce gli spazi prima della punteggiatura.

7. Limiti noti per fonte

Fonte	Limite	Mitigazione
FEVER	scorciatoie solo-claim ("not", "never" → REFUTES); NEI accoppiati in automatico (negativi facili)	quota limitata; esperimento con filtro delle negazioni; test "solo claim"
ANLI	premesse corte	combinarlo con fonti a documenti lunghi
SummEdits	alcune modifiche opinabili	Brier al posto della CE se il rumore pesa (04_loss.md)
AVeriTeC	formato domanda–risposta, 70% negativi	quota piccola, nessun bilanciamento forzato
VitaminC	evidenza di 1–2 frasi	campione, non tutto

Criteri per passare allo step successivo

- Tutte le fonti scelte presenti in `data/finetune/raw/` con le statistiche.
- Decontaminazione eseguita e numeri registrati.
- Campione letto a mano senza sorprese.